

# Module 20: Packages and Modules

---

Packages organise names. The classpath tells the JVM where to look. Modules add enforcement on top of both.

Most of what you need here is practical: enough to read a build file, diagnose a startup failure, and understand why `--add-opens` exists.

## What You'll Learn

---

- The package-to-directory rule, and package-private as a real access level
- The classpath, and the two startup errors that are not the same thing
- Module import declarations, and when not to use them
- Building a jar, and what a module adds that a jar cannot

## Coming From Another Language

---

|                      | Python                       | Java                                                      |
|----------------------|------------------------------|-----------------------------------------------------------|
| Namespace            | module = file, package = dir | package, declared in the file                             |
| Layout enforced      | by import path               | <b>by the compiler</b>                                    |
| Search path          | <code>sys.path</code>        | classpath or module path                                  |
| Import everything    | <code>from x import *</code> | <code>import x.*</code> , or <code>import module m</code> |
| Privacy across files | underscore convention        | package-private, enforced                                 |
| Distribution unit    | wheel                        | jar                                                       |

The enforced-layout row is the one that catches people. In Java the `package` statement and the directory path **must** match, and the compiler will tell you.

## The Lesson

---

### Packages

```
package com.example.util;    ->    com/example/util/Greeter.java
```

The compiler uses the directory to find sources it was not given, and the JVM uses it to find classes at runtime.

`Packages.java` builds a small two-package project in a temporary directory, compiles it with the real `javac`, and runs it:

```
javac exit code: 0
produced: [classes/com/example/app/Main.class, classes/com/example/util/Greeter.class]
<< Hello, world >>
```

**Naming:** reverse domain name, all lower case. The reversal exists so two organisations cannot collide.

**Never use the default package for real code.** A class with no `package` statement cannot be imported by any class that has one, so it is unusable from anywhere organised.

**Package-private is a real access level**, and the one people forget:

| Level                  | Reach                                 |
|------------------------|---------------------------------------|
| <code>private</code>   | this class                            |
| <code>(none)</code>    | <b>this package</b>                   |
| <code>protected</code> | this package plus subclasses anywhere |
| <code>public</code>    | everyone                              |

It is how collaborating classes share something without exposing it to the world.

## The classpath

A list of directories and jars, **searched in order**:

```
java -cp classes:lib/one.jar:lib/two.jar com.example.Main
java -cp "classes;lib/*" com.example.Main      (Windows uses ;)
```

**The two errors are not the same.** From `TheClasspath.java`:

**ClassNotFoundException** is a checked exception. Something asked for a class *by name* at runtime and it was not found. Usually reflection, a JDBC driver, or a framework loading a configured class.

**NoClassDefFoundError** is an Error. The class was present at compile time and is missing or unusable now. And here is the case people misdiagnose:

```
first attempt: ExceptionInInitializerError, caused by java.lang.IllegalStateException:
the static initialiser failed
second attempt: NoClassDefFoundError: Could not initialize class TheClasspath$Broken
```

A class whose static initialiser threw reports the real cause **once**. Every attempt after that says `NoClassDefFoundError` with no hint at all.

If you see `NoClassDefFoundError` for a class you are certain is on the classpath, scroll **up** the log. The real failure was reported earlier and looked unrelated.

**The failure mode nobody expects:** two jars containing the same class. The classpath is searched in order, the first wins, the second is silently ignored. Symptoms are a method that does not exist, or behaviour from a version you thought you had replaced. This is the problem the module system was built to solve.

## Module import declarations

JEP 511, final in JDK 25:

```
import module java.base;
```

One line imports every public package that module exports. From `ModuleImports.java`, with no other import:

```
names:    [ada, grace, alan]
path:     /tmp/example.txt
time:     2026-09-23
```

It is a **compile-time convenience**. The bytecode is identical.

**Use it** for scripts, single-file programs and teaching examples. **Do not use it** in a real codebase: explicit imports document what a file depends on, so a reviewer can see when a file starts reaching somewhere new. It also makes ambiguity possible when two modules export the same simple name.

## Jars and modules

From `JarsAndModules.java`:

```
jar --create --file tool.jar --main-class com.example.tool.Tool -C classes .
```

```
META-INF/MANIFEST.MF
com/example/tool/Tool.class
```

`MANIFEST.MF` is what `--main-class` wrote, which is how `java -jar` knows where to start.

A **module** adds a declaration:

```
module com.example.tool {
    requires java.logging;    // what it needs
    exports com.example.tool; // what others may use
}
```

Three things a plain jar cannot do:

1. **Strong encapsulation.** A package that is not exported is inaccessible from outside, even by reflection. `public` stops meaning universally reachable.
2. **Dependencies checked at startup.** A missing `requires` fails when the JVM launches, not as a `NoClassDefFoundError` an hour into production.
3. **No split packages.** Two modules cannot contain the same package, which is exactly the silent shadowing described above.

**Why most projects still use the classpath:** modularising means every dependency must be modular too, or be treated as an automatic module, which gets you name checking without encapsulation.

**Where most developers actually meet the module system:** an `InaccessibleObjectException` from a library reflecting into `java.base`, fixed with `--add-opens`. The practical minimum is knowing why that flag exists: `java.base` does not export its internals to you, and `--add-opens` overrides that.

## The tools

```
jar --list --file x.jar           # see inside
jar --describe-module --file x.jar # is it modular
jdeps --list-deps x.jar          # what it needs
javadoc -d docs src/**/*.java   # generate documentation
```

## Run It

```
java modules/20-packages-and-modules/examples/Packages.java
java modules/20-packages-and-modules/examples/TheClasspath.java
java modules/20-packages-and-modules/examples/ModuleImports.java
java modules/20-packages-and-modules/examples/JarsAndModules.java

./scripts/verify-examples.sh modules/20-packages-and-modules
```

Two of these build a real packaged project in a temporary directory, compile it with `javac`, and delete it afterwards. Nothing is left behind.

## Common Mistakes

**Package statement not matching the directory.** The compiler says so; believe it rather than fighting it.

**Using the default package.** Nothing with a package can import you.

**Making everything `public` by reflex.** Package-private exists and is usually right for collaborating classes.

**Diagnosing `NoClassDefFoundError` as a missing jar** when a static initialiser failed earlier.

**Two versions of a library on the classpath.** First wins, silently.

`import module` in production code. It hides what a file depends on.

**Adding `--add-opens` without understanding it.** It is a hole punched in another module's encapsulation. Sometimes necessary, never free.

## Key Takeaways

---

- **The package statement and the directory must match**, and the compiler enforces it.
- **Package-private is the default access level**, not an absence of one.
- **The classpath is searched in order**, so a duplicate class is shadowed silently.
- `ClassNotFoundException` means asked for by name; `NoClassDefFoundError` means it was there at compile time. A failed static initialiser causes the second and hides the first.
- `import module` is a compile-time convenience for scripts, not for codebases.
- **Modules add encapsulation, startup-time dependency checking and no split packages**, which is why `--add-opens` exists.

## Homework

---

[homework/README.md](#)

Build a two-package project from scratch, jar it, then diagnose three startup failures. Reference solution in [solutions/20-packages-and-modules/](#).